1 2 9 0

UNIVERSIDADE D
COIMBRA

Licenciatura de Engenharia Informática
Base de Dados

Sistema de Gestão Universitária

Índice

Contactos e Membros 3

1.Manual de Instalação 3

1.1. Pré-requisitos 3

1.2. Dependências 3

1.3. Configuração da Base de Dados 3

1.4. Variáveis de Ambiente 4

1.5. Executar o Projeto 4

2. Manual de Uso (Com Postman) 4

2.1. Configuração Geral no Postman 4

2.2. Endpoints 5

2.2.1. Login de Utilizador 5

2.2.2. Registar Estudante 5

2.2.3. Registar Staff 6

2.2.4. Registar Instrutor 6

2.2.5. Inscrever Estudante em Degree 7

2.2.6. Inscrever Estudante em Atividade Extracurricular 7

2.2.7. Inscrever Estudante em Edição de Unidade Curricular 8

2.2.8. Submeter Notas 8

2.2.9. Detalhes do Estudante 9

2.2.10. Detalhes da Degree 9

2.2.11. Top 3 Estudantes 9

2.2.12. Melhor Estudante por Distrito 10

2.2.13. Relatório Mensal 10

2.2.14. Eliminar Estudante 10

3. Descrição de Cada Ponto (Endpoint) e Como Funcionam 10

3.1. Estrutura Geral e Autenticação 10

3.2. Descrição Detalhada dos Endpoints 11

Plano de desenvolvimento 18

Contactos e Membros

Pedro Diniz – uc2023214607@student.uc.pt

Francisco Gouveia – uc2023214517@student.uc.pt

Tiago Teixeira – uc2023224647@student.uc.pt

1.Manual de Instalação

1.1. Pré-requisitos

- **Python:** Versão 3.x.
- **PostgreSQL:** Uma instância do PostgreSQL em execução.
- **pip:** Gestor de pacotes Python.

1.2. Dependências

O projeto utiliza as seguintes bibliotecas Python:

- flask: Para a criação da API web.
- psycopg2-binary: Para a ligação à base de dados PostgreSQL.
- PyJWT: Para a geração e validação de JSON Web Tokens.
- python-dotenv: Para gerir variáveis de ambiente.

Estas dependências podem ser instaladas utilizando o pip:

pip install flask psycopg2-binary PyJWT python-dotenv

1.3. Configuração da Base de Dados

O projeto requer uma base de dados PostgreSQL. As queries SQL no código sugerem a existência de tabelas como person, student, administrative_staff,

coordinator_instructor, assistent_instructor, student_degree, degree_course, course, course_edition, edition, grade, period, student_extracurricular_activities, student_class, timetable, e assistent_instructor_course_edition. Certifique-se de que estas tabelas e as suas respetivas colunas, incluindo a extensão pgcrypto para a função crypt e gen_salt, estão configuradas na sua base de dados PostgreSQL.

1.4. Variáveis de Ambiente

Crie um ficheiro .env na raiz do projeto com as seguintes variáveis de ambiente, preenchidas com os seus respetivos valores:

Fragmento do código

DB_USER=seu_usuario_db

DB_PASSWORD=sua_senha_db

DB_HOST=seu_host_db

DB_PORT=sua_porta_db

DB_NAME=seu_nome_db

JWT_SECRET=sua_chave_secreta_jwt

O JWT_SECRET é crucial para a segurança da autenticação baseada em tokens.

1.5. Executar o Projeto

Após a instalação das dependências e configuração das variáveis de ambiente, pode executar o servidor Flask:

python project_api.py

Por defeito, a API estará disponível em http://127.0.0.1:8080. O logging será registado no ficheiro log_file.log e na consola.

2. Manual de Uso (Com Postman)

2.1. Configuração Geral no Postman

- **Base URL:** http://127.0.0.1:8080/dbproj
- **Headers:**
    - Content-Type: application/json para pedidos POST e PUT que enviam dados JSON.

- Authorization: Para endpoints protegidos, o valor deste header deve ser o token JWT obtido após o login (e.g., eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...).

2.2. Endpoints

2.2.1. Login de Utilizador

- **Método:** PUT
- **URL:** /user
- **Corpo (JSON):**

JSON

```
{
  "username": "seu_username",
  "password": "sua_password"
}
```

- **Descrição:** Autentica um utilizador. Se as credenciais forem válidas, retorna um token JWT.

2.2.2. Registar Estudante

- **Método:** POST
- **URL:** /register/student
- **Headers:** Authorization: <token_staff>
- **Corpo (JSON):**

JSON

```
{
  "username": "nome_estudante",
  "cc": "cc_estudante",
  "age": idade_estudante,
  "district": "distrito_estudante",
  "email": "email_estudante@example.com",
  "password": "password_estudante",
```

"student_number": "numero_estudante",

"account_value": valor_conta_estudante

}

- **Descrição:** Regista um novo estudante. Requer um token de um membro do 'staff'.

2.2.3. Registar Staff

- **Método:** POST
- **URL:** /register/staff
- **Headers:** Authorization: <token_staff>
- **Corpo (JSON):**

JSON

{

"username": "nome_staff",

"cc": "cc_staff",

"age": idade_staff,

"district": "distrito_staff",

"email": "email_staff@example.com",

"password": "password_staff",

"staff_number": "numero_staff"

}

- **Descrição:** Regista um novo membro do staff. Requer um token de um membro do 'staff'.

2.2.4. Registar Instrutor

- **Método:** POST
- **URL:** /register/instructor
- **Headers:** Authorization: <token_staff>
- **Corpo (JSON):**

JSON

```
{
  "username": "nome_instrutor",
  "cc": "cc_instrutor",
  "age": idade_instrutor,
  "district": "distrito_instrutor",
  "email": "email_instrutor@example.com",
  "password": "password_instrutor",
  "type": "coordenador", // ou "assistente"
  "instructor_number": "numero_instrutor"
}
```

- **Descrição:** Regista um novo instrutor (coordenador ou assistente). Requer um token de um membro do 'staff'.

2.2.5. Inscrever Estudante em Degree

- **Método:** POST
- **URL:** /enroll_degree/<degree_id> (substituir <degree_id> pelo ID da licenciatura/mestrado)
- **Headers:** Authorization: <token_staff>
- **Corpo (JSON):**

JSON

```
{
  "student_id": id_estudante,
  "date": "YYYY-MM-DD"
}
```

- **Descrição:** Inscreve um estudante numa licenciatura/mestrado. Requer um token de um membro do 'staff'.

2.2.6. Inscrever Estudante em Atividade Extracurricular

- **Método:** POST
- **URL:** /enroll_activity/<activity_id> (substituir <activity_id> pelo ID da atividade)
- **Headers:** Authorization: <token_estudante>

- **Corpo (JSON):** (Não necessita de corpo)
- **Descrição:** Inscreve um estudante (identificado pelo token) numa atividade extracurricular.

2.2.7. Inscrever Estudante em Edição de Unidade Curricular

- **Método:** POST
- **URL:** /enroll_course_edition/<course_edition_id> (substituir <course_edition_id> pelo ID da edição da UC)
- **Headers:** Authorization: <token_estudante>
- **Corpo (JSON):**

JSON

```
{
  "classes": [id_turma1, id_turma2] // IDs das turmas (pelo menos uma)
}
```

- **Descrição:** Inscreve um estudante (identificado pelo token) numa ou mais turmas de uma edição de unidade curricular. Verifica a capacidade da edição e se o estudante está inscrito na licenciatura/mestrado que contém a UC.

2.2.8. Submeter Notas

- **Método:** POST
- **URL:** /submit_grades/<course_edition_id> (substituir <course_edition_id> pelo ID da edição da UC)
- **Headers:** Authorization: <token_instrutor_coordenador>
- **Corpo (JSON):**

JSON

```
{
  "year": ano, // e.g., 2024
  "semestre": semestre, // e.g., 1 ou 2
  "grades": [
    {"student_id": id_estudante1, "grade": nota1},
    {"student_id": id_estudante2, "grade": nota2}
  ],
```

"period_type": "tipo_periodo", // e.g., "Normal", "Recurso"

"date": "YYYY-MM-DD" // data da avaliação

}

- **Descrição:** Permite a um instrutor coordenador da unidade curricular submeter as notas dos estudantes. Utiliza isolamento SERIALIZABLE.

2.2.9. Detalhes do Estudante

- **Método:** GET
- **URL:** /student_details/<student_id> (substituir <student_id> pelo ID do estudante)
- **Headers:** Authorization: <token_staff>
- **Descrição:** Retorna detalhes das médias das unidades curriculares de um estudante nas suas inscrições ativas. Utiliza isolamento READ COMMITTED.

2.2.10. Detalhes da Degree

- **Método:** GET
- **URL:** /degree_details/<degree_id> (substituir <degree_id> pelo ID da licenciatura/mestrado)
- **Headers:** Authorization: <token_staff>
- **Descrição:** Retorna detalhes sobre as unidades curriculares de uma licenciatura/mestrado, focando na edição mais recente de cada UC. Inclui contagem de inscritos, aprovados, coordenador e assistentes. Utiliza isolamento READ COMMITTED.

2.2.11. Top 3 Estudantes

- **Método:** GET
- **URL:** /top3
- **Headers:** Authorization: <token_staff>
- **Descrição:** Retorna os 3 melhores estudantes com base na média das notas do ano corrente. Inclui nome, média, lista de notas e atividades extracurriculares. Utiliza isolamento SERIALIZABLE.

2.2.12. Melhor Estudante por Distrito

- **Método:** GET
- **URL:** /top_by_district
- **Headers:** Authorization: <token_staff>
- **Descrição:** Retorna o estudante com a média mais alta do ano corrente para cada distrito. Utiliza isolamento SERIALIZABLE.

2.2.13. Relatório Mensal

- **Método:** GET
- **URL:** /report
- **Headers:** Authorization: <token_staff>
- **Descrição:** Gera um relatório mensal dos últimos 12 meses, mostrando a edição da unidade curricular com mais estudantes aprovados em cada mês. Inclui o número de aprovados e avaliados nessa edição. Utiliza isolamento SERIALIZABLE.

2.2.14. Eliminar Estudante

- **Método:** DELETE
- **URL:** /delete_details/<student_id> (substituir <student_id> pelo ID do estudante)
- **Headers:** Authorization: <token_staff>
- **Descrição:** Remove um estudante da tabela student. Requer um token de um membro do 'staff'. (Nota: Dependendo da configuração da base de dados, a pessoa associada pode ou não ser eliminada em cascata).

3. Descrição de Cada Ponto (Endpoint) e Como Funcionam

3.1. Estrutura Geral e Autenticação

- **Flask App:** O projeto é uma aplicação Flask.
- **Base de Dados:** Utiliza PostgreSQL, com ligações geridas pela função db_connection().
- **Autenticação JWT:**
    - A função check_token (token, type) valida tokens JWT. Verifica a assinatura, expiração e se o user_id no payload do token

corresponde a um utilizador do tipo especificado (student, staff, instructor) na base de dados.

    - O decorador @token_required protege os endpoints, garantindo que um token é fornecido no header Authorization.

- **Criação de Pessoa (create_person):**
    - Função auxiliar para criar um registo na tabela person.
    - Recebe dados como nome de utilizador, CC, idade, distrito, email e password.
    - A password é cifrada usando crypt com gen_salt('bf').
    - Retorna o id da pessoa criada.

3.2. Descrição Detalhada dos Endpoints

- **PUT /dbproj/user (login_user)**
    - **Funcionalidade:** Permite que um utilizador existente faça login.
    - **Processo:**
        1. Recebe username e password do corpo do pedido.
        2. Verifica na tabela person se existe um utilizador com o username fornecido e se a password corresponde à password cifrada na base de dados.
        3. Se as credenciais forem válidas, gera um token JWT contendo o user_id e um tempo de expiração de 60 minutos.
        4. Retorna o token JWT.
- **POST /dbproj/register/student (register_student)**
    - **Funcionalidade:** Regista um novo estudante no sistema.
    - **Permissões:** Requer um token de autenticação de um utilizador do tipo 'staff'.
    - **Processo:**
        1. Valida o token do 'staff'.
        2. Recebe os dados do estudante (incluindo username, email, password, etc., para a tabela person, e student_number, account_value para a tabela student) do corpo do pedido.
        3. Chama create_person para inserir os dados básicos na tabela person.

4. Se a criação da pessoa for bem-sucedida, insere os dados específicos do estudante (student_number, account_value, person_id) na tabela student.
5. Retorna o person_id do novo estudante e o seu student_number.

- **POST /dbproj/register/staff (register_staff)**
  - **Funcionalidade:** Regista um novo membro da equipa administrativa (staff).
  - **Permissões:** Requer um token de autenticação de um utilizador do tipo 'staff'.
  - **Processo:**
    1. Valida o token do 'staff'.
    2. Recebe os dados do staff (incluindo username, email, password, etc., para a tabela person, e staff_number para a tabela administrative_staff) do corpo do pedido.
    3. Chama create_person.
    4. Insere os dados (staff_number, person_id) na tabela administrative_staff.
    5. Retorna o person_id e staff_number.
- **POST /dbproj/register/instructor (register_instructor)**
  - **Funcionalidade:** Regista um novo instrutor, que pode ser do tipo 'coordenador' ou 'assistente'.
  - **Permissões:** Requer um token de autenticação de um utilizador do tipo 'staff'.
  - **Processo:**
    1. Valida o token do 'staff'.
    2. Recebe os dados do instrutor (para person e type, instructor_number).
    3. Chama create_person.
    4. Com base no type ('coordenador' ou 'assistente'), insere os dados na tabela coordinator_instructor ou assistent_instructor.
    5. Retorna o person_id e instructor_number.
- **POST /dbproj/enroll_degree/<degree_id> (enroll_degree)**

- **Funcionalidade:** Permite que um 'staff' inscreva um estudante numa licenciatura/mestrado (degree).
- **Permissões:** Requer um token de 'staff'.
- **Processo:**
  1. Valida o token do 'staff'.
  2. Recebe student_id e date do corpo do pedido JSON, e degree_id do URL.
  3. Insere um novo registo na tabela student_degree, marcando a inscrição como active = true.

- **POST /dbproj/enroll_activity/<activity_id> (enroll_activity)**
  - **Funcionalidade:** Permite que um estudante se inscreva numa atividade extracurricular.
  - **Permissões:** Requer um token de 'student'. O ID do estudante é extraído do token.
  - **Processo:**
    1. Valida o token do estudante e obtém o student_id.
    2. Recebe activity_id do URL.
    3. Insere um registo na tabela student_extracurricular_activities.
- **POST /dbproj/enroll_course_edition/<course_edition_id> (enroll_course_edition)**
  - **Funcionalidade:** Permite que um estudante se inscreva numa edição de uma unidade curricular.
  - **Permissões:** Requer um token de 'student'. O ID do estudante é extraído do token.
  - **Processo:**
    1. Valida o token do estudante e obtém o student_id.
    2. Recebe course_edition_id do URL e uma lista de classes (IDs das turmas) do corpo JSON.
    3. Para cada class_id fornecido:
       - Verifica se a capacidade (capacity) da course_edition não foi excedida pelo número de inscritos (enrolled_count).
       - Verifica se o estudante está ativo (active = TRUE) numa licenciatura/mestrado (student_degree) que

inclua a unidade curricular (degree_course) desta course_edition.

        - Se ambas as condições forem satisfeitas, insere o estudante na turma (student_class).

- **POST /dbproj/submit_grades/<course_edition_id> (submit_grades)**
    - **Funcionalidade:** Permite a um instrutor (coordenador) submeter as notas dos estudantes para uma edição de unidade curricular.
    - **Permissões:** Requer um token de 'instructor'. O ID do instrutor é extraído do token e deve corresponder ao coordinator_instructor_professor_person_id da course_edition.
    - **Processo:**
        1. Valida o token do instrutor e obtém o instructor_id.
        2. Obtém course_edition_id do URL.
        3. Verifica se o instructor_id corresponde ao coordenador da course_edition (utiliza FOR UPDATE para bloquear a linha durante a verificação).
        4. Recebe year, semestre, grades (lista de objetos com student_id e grade), period_type e date do corpo JSON.
        5. Inicia uma transação com nível de isolamento SERIALIZABLE.
        6. Para cada nota na lista grades, insere-a na tabela grade, identificando o período correto com base em period_type, semestre e year.
        7. Faz commit da transação.
- **GET /dbproj/student_details/<student_id> (student_details)**
    - **Funcionalidade:** Obtém detalhes das médias das unidades curriculares de um estudante.
    - **Permissões:** Requer um token de 'staff'.
    - **Processo:**
        1. Valida o token do 'staff'.
        2. Recebe student_id do URL.
        3. Inicia uma transação com nível de isolamento READ COMMITTED.
        4. Consulta a base de dados para obter o nome da unidade curricular, ID da edição, ano da edição e a média das notas do estudante (AVG(g.grade)) para cada unidade curricular

em que o estudante está inscrito numa licenciatura/mestrado ativa (sd.active IS TRUE).

5. Os resultados são agrupados por edição de UC, nome da UC e ano da edição.
6. Retorna os detalhes em formato JSON.

- **GET /dbproj/degree_details/<degree_id> (degree_details)**
  - **Funcionalidade:** Obtém detalhes das unidades curriculares pertencentes a uma licenciatura/mestrado.
  - **Permissões:** Requer um token de 'staff'.
  - **Processo:**
    1. Valida o token do 'staff'.
    2. Recebe degree_id do URL.
    3. Inicia uma transação com nível de isolamento READ COMMITTED.
    4. Consulta a base de dados para obter, para cada unidade curricular (course) associada à licenciatura/mestrado (degree_id):
       - Código e nome.
       - ID e ano da edição mais recente (MAX(e2.edition_year)).
       - Capacidade da edição (capacity).
       - Número de estudantes inscritos (enrolled_count), calculado pela contagem distinta de estudantes em student_class associados aos horários (timetable) da edição.
       - Número de estudantes aprovados (approved_students), contado a partir da tabela grade (nota >= 10).
       - Número do coordenador da edição.
       - Lista de IDs dos professores assistentes (assistents) associados à edição.
    5. Os resultados são agregados em formato JSON.

- **GET /dbproj/top3 (top3_students)**
  - **Funcionalidade:** Lista os 3 melhores estudantes com base na média das notas obtidas no ano corrente.

- **Permissões:** Requer um token de 'staff'.
- **Processo:**
    1. Valida o token do 'staff'.
    2. Inicia uma transação com nível de isolamento SERIALIZABLE.
    3. Calcula a média das notas (AVG(g.grade)) para cada estudante, considerando apenas as notas do ano corrente (DATE_PART('year', g.grade_date) = DATE_PART('year', CURRENT_DATE)).
    4. Obtém o nome do estudante da tabela person.
    5. Agrega as notas individuais do ano corrente para cada estudante em formato JSON.
    6. Agrega as atividades extracurriculares de cada estudante em formato JSON.
    7. Ordena os estudantes pela média de forma descendente e limita o resultado a 3.
    8. Retorna os dados dos 3 melhores estudantes.

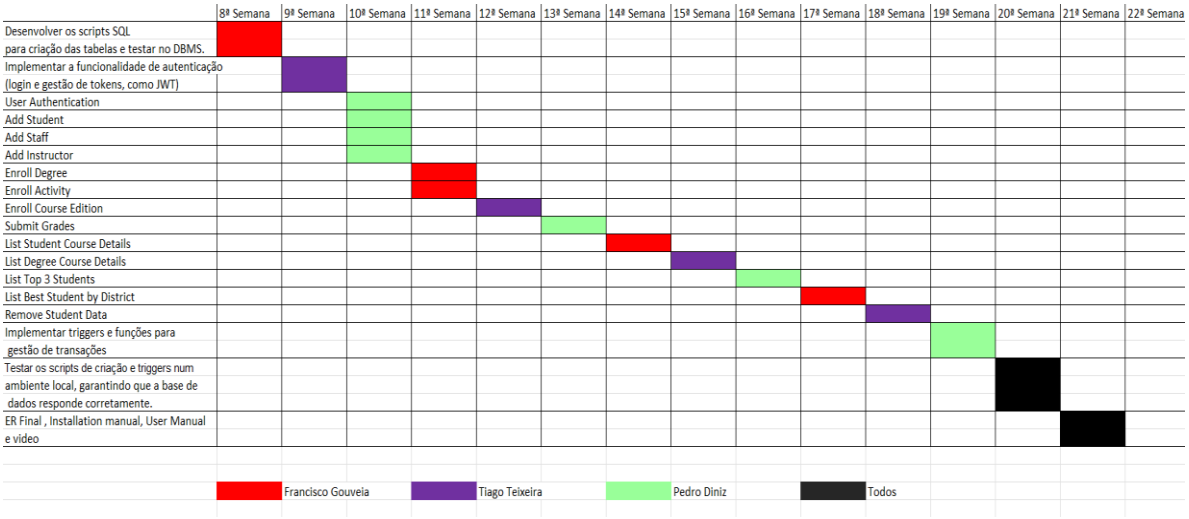
- **GET /dbproj/top_by_district (top_by_district)**
    - **Funcionalidade:** Lista o melhor estudante (maior média de notas no ano corrente) de cada distrito.
    - **Permissões:** Requer um token de 'staff'.
    - **Processo:**
        1. Valida o token do 'staff'.
        2. Inicia uma transação com nível de isolamento SERIALIZABLE.
        3. Calcula a média das notas (avg_grade) para cada estudante no ano corrente.
        4. Determina a média máxima (max_avg) para cada distrito.
        5. Seleciona os estudantes cuja média (avg_grade) é igual à média máxima (max_avg) do seu respetivo distrito.
        6. Retorna o distrito, a média e o ID do estudante para cada um desses estudantes, ordenado por distrito.

- **GET /dbproj/report (monthly_report)**

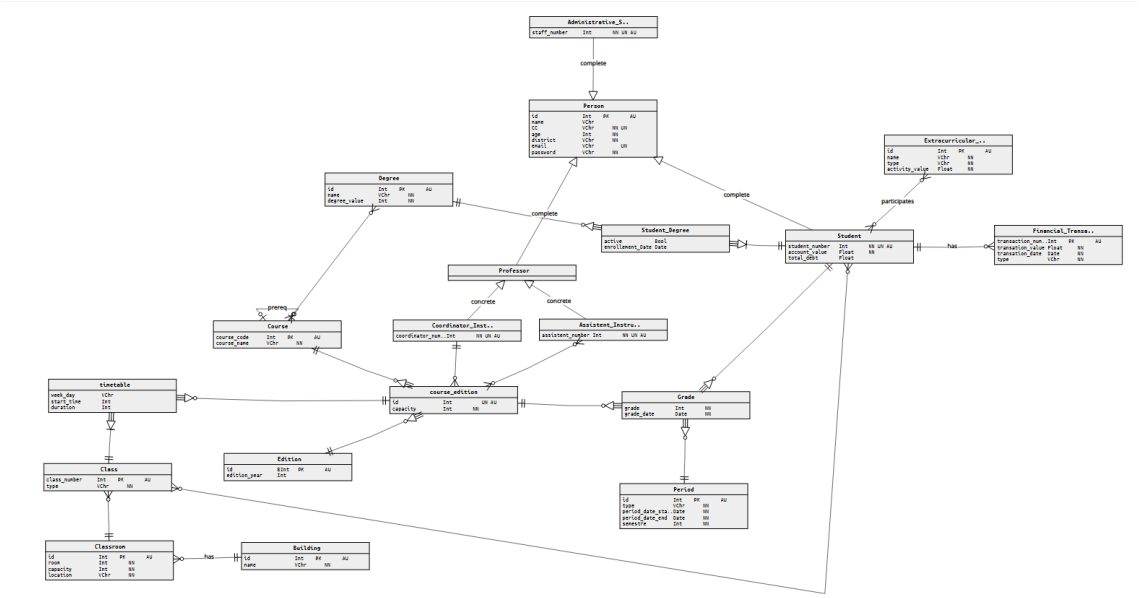
- **Funcionalidade:** Gera um relatório mensal para os últimos 12 meses, indicando, para cada mês, a edição de unidade curricular com o maior número de estudantes aprovados.
- **Permissões:** Requer um token de 'staff'.
- **Processo:**
  1. Valida o token do 'staff'.
  2. Inicia uma transação com nível de isolamento SERIALIZABLE.
  3. Para cada mês nos últimos 12 meses (g.grade_date >= (CURRENT_DATE - INTERVAL '12 months')):
     - Calcula o número de estudantes aprovados (COUNT(CASE WHEN g.grade >= 10 THEN 1 END)) e o total de estudantes avaliados (COUNT(*)) por edição de UC.
     - Usando DISTINCT ON (DATE_PART('year', g.grade_date), DATE_PART('month', g.grade_date)), seleciona a edição de UC com o maior número de aprovados para esse mês/ano.
  4. Retorna o mês, ID da edição, nome da UC, número de aprovados e avaliados para cada entrada do relatório, ordenado por mês.

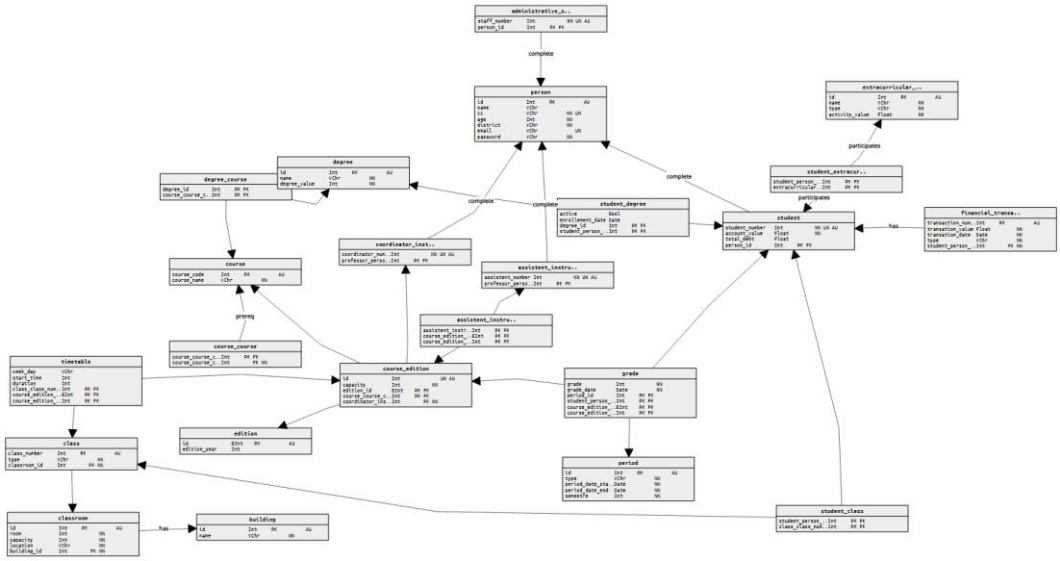
- **DELETE /dbproj/delete_details/<student_id> (delete_student)**
  - **Funcionalidade:** Remove um estudante do sistema.
  - **Permissões:** Requer um token de 'staff'.
  - **Processo:**
    1. Valida o token do 'staff'.
    2. Recebe student_id do URL.
    3. Executa uma query DELETE na tabela student para remover o registo do estudante com o person_id correspondente.

Plano de desenvolvimento



Diagramas




administrative_a..
stuff_number Int NN UN AU
person_id Int PK FK
complete
person
id Int PK AU
name VChr
lo VChr NN UN
age Int NN
district VChr NN
email VChr UN
password VChr NN
extracurricular...
id Int PK AU
name VChr NN
type VChr NN
activity_value Float NN
participates
degree
id Int PK AU
name VChr NN
degree_value Int NN
degree_course
degree_id Int PK FK
course_course_c.. Int PK FK
complete
complete
complete
student_extracur..
student_person_.. Int PK FK
extracurricular.. Int PK FK
participates
student_degree
active Bool
enrollement_date Date
degree_id Int PK FK
student_person_..Int PK FK
student
student_number Int NN UN AU
account_value Float NN
total_ects Float
person_id Int PK FK
has
financial_transa..
transaction_num..Int PK AU
transaction_value Float NN
transaction_date Date NN
type VChr NN
student_person_.. Int PK NN
coordinator_teat..
coordinator_num..Int NN UN AU
professor_perso..Int PK FK
course
course_code Int PK AU
course_name VChr NN
assistent_teatru..
assistent_number Int NN UN AU
professor_perso..Int PK FK
prereq
assistent_teatru..
assistent_teatr..Int PK FK
course_edition_.. BInt PK FK
course_edition_..Int PK FK
course_course
course_course_c..Int PK FK
course_course_c..Int PK NN
timetable
week_day VChr
start_time Int
duration Int
class_class_num..Int PK FK
course_edition_..BInt PK FK
course_edition_..Int PK FK
course_edition
id Int UN AU
capacity Int NN
course_id Int PK FK
course_course_c..Int PK FK
coordinator_id..Int PK NN
grade
grade Int NN
grade_date Date NN
period_id Int PK FK
student_person_ Int PK FK
course_edition_ BInt PK FK
course_edition_ Int PK FK
edition
id BInt PK AU
edition_year Int
class
class_number Int PK AU
type VChr NN
classroom_id Int FK NN
period
id Int PK AU
type VChr NN
period_date_sta..Date NN
period_date_end Date NN
semestre Int NN
classroom
id Int PK AU
room Int NN
capacity Int NN
location VChr NN
building_id Int FK NN
has
building
id Int PK AU
name VChr NN
student_class
student_person_..Int PK FK
class_class_num..Int PK FK